

CS 599: Foundations of Deep Learning

Assignment #00011

December 17, 2024

Course Policy

Carefully read all the instructions below before you start working on the assignment, and before you make a submission.

- All Problems should be coded in Tensorflow 2 or above
- Please typeset your submissions in L^AT_EX, give maximum explanation for each subproblems.
- Assignments are due at the end of the day at 11:59 pm on the due date given on the web Portal.
- No single line answers are accepted in the submission.
- Late assignments will suffer 50 percent loss after the first day and all loss after the second.
- All source materials must be cited. The University Academic Code of Conduct will be strictly enforced.
- We will be creating Canvas submission page for this. You have to submit python file[no ipython allowed] given in gitrepo along with your response pdf.

Problem 1: Implementing various Normalization using basic tensorflow ops (3+3+2=8 points)

1 Batch Normalization [1]: The basic Idea

Batch normalization(BN) normalizes activations in our model across the mini-batches. For each input feature available in the model, BN then computes the mean and variance of those features within mini-batch. Then it subtracts the mean and divides the input features by mini-batch standard deviations. **So what does this do?** BN restricts the activations to have 0 mean and unit variance/std. Now let's think about few problems associated with this?

How will these approach work, when we have relu?

If increase in weight magnitude leads to better convergence. To address this BN adds two learnable parameters: First is mean of activation and second is magnitude of activation. Then BN rescales the normalized activations/post-activation and then goes on adding a constant. This ensures that expressiveness ability of neural network holds.

Let's get into details:

x_i = input features within a mini-batch(MB) of size N

z_i = output after applying BN operations

γ and β are learnable parameters

$$\begin{aligned}
\mu_{MB} &\leftarrow \frac{1}{N} \sum_{i=1}^N (x_i) && // \text{calculate mini-batch mean} \\
\sigma_{MB}^2 &\leftarrow \frac{1}{N} \sum_{i=1}^N (x_i - \mu_{MB})^2 && // \text{calculate mini-batch variance} \\
\hat{x}_i &\leftarrow (x_i - \mu_{MB}) / \sqrt{\sigma_{MB}^2 + \epsilon} && // \text{normalize data} \\
z_i &\leftarrow \gamma \hat{x}_i + \beta && // \text{Finally scale and shift.}
\end{aligned}$$

2 Weight Normalization [3]

Standard way of calculating post-activation for any given input is given by: $Y = \phi(W.x + b)$ Y is output, W is weight matrix, b is biases and x is input. Now we will do reparameterizing of each weight w in terms of parameter v and a scalar parameter g. How to achieve this:

$$w = (g/||v||)v$$

$||v||$ denotes the euclidean norm of v, and v is k-dimensional vector. If we fix the euclidean norm of w then we get $||w|| = g$.

[Note:- SGD step is now performed on new parameters v and g]

3 Layer Normalization[2]

Layer norm normalizes the inputs across the features. Whereas batch norm normalizes the input feature across mini batches.

Let's modify layer norm original equation and compare with batch norm to understand exact difference.

Batch Norm

Steps for batchnorm:

$$\begin{aligned}
\text{Step1: } \mu_j &\leftarrow \frac{1}{N} \sum_{i=1}^N (x_{ij}) && // \text{calculate mini-batch mean} \\
\text{Step2: } \sigma_j^2 &\leftarrow \frac{1}{N} \sum_{i=1}^N (x_{ij} - \mu_j)^2 && // \text{calculate mini-batch variance} \\
\text{Step3: } \hat{x}_{ij} &\leftarrow (x_{ij} - \mu_j) / \sqrt{\sigma_j^2 + \epsilon} && // \text{normalize data} \\
\text{Step4: } z_{ij} &\leftarrow \gamma \hat{x}_{ij} + \beta && // \text{Finally scale and shift.}
\end{aligned}$$

x_{ij} is the i,jth element of input, which also represents batch and feature respectively.

LayerNorm

Steps for LayerNorm:

$$\begin{aligned}
\text{Step1: } \mu_i &\leftarrow \frac{1}{N} \sum_{j=1}^N (x_{ij}) \\
\text{Step2: } \sigma_i^2 &\leftarrow \frac{1}{N} \sum_{j=1}^N (x_{ij} - \mu_i)^2 \\
\text{Step3: } \hat{x}_{ij} &\leftarrow (x_{ij} - \mu_i) / \sqrt{\sigma_i^2 + \epsilon} \\
\text{Step4: } z_{ij} &\leftarrow \gamma \hat{x}_{ij} + \beta
\end{aligned}$$

Things to do:

- Modify the file provided[You can include functions but can't delete any]
- You will be working with Fashion MNIST or cifar10[Depends on resources you have]. I provide you the flexibility to chose the dataset for this problem[either fashion or cifar]
- You can use keras or tf.layers to declare CNN and maxpooling layer. I am giving freedom for this.
- Backward Pass must use gradient tape method.
- Create forward pass for all normalization approaches using basic tensorflow ops and formulation provided.
- You should have WeightNorm, BatchNorm and LayerNorm three different functions. Use them in your original forward pass.
- Compare Results with and without Normalization.
- Compare your normalization function with tensorflow Norm functions. Is there any difference in performance beside floating point error? If so check your backward pass.[You should get comparable gradients]. Calculate the difference between your Normalization function and one with tensorflow function.
- Report your findings, based on your experiments which one is good and why?. Also explain why LayerNorm is better than batchNorm?

References

1. Ioffe, S., and Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167* (2015).
2. Lei Ba, J., Kiros, J. R., and Hinton, G. E. Layer normalization. *arXiv preprint arXiv:1607.06450* (2016).
3. Salimans, T., and Kingma, D. P. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In *Advances in Neural Information Processing Systems* (2016), pp. 901–909.